

ハーネスエンジニアリングの 概要と設計思想



渋谷 優介 @sergicalsix

ALGOMATIC
AI Transformation

ハーネスエンジニアリングとは

ハーネスとは

文献・事例	ハーネスの位置付け	キーワード
Biderman(2024)	評価を成立させる基盤	<i>evaluation harness</i>
Xu(2024)	LLMに行動手段を与えるランタイム・周辺環境	<i>digital worker environment</i>
Zhang(2025)	LLMに追加可能なモジュール群	<i>modular harness design</i>
Bui(2026)	実行のオーケストレーション層	<i>runtime orchestration layer</i>
Anthropic(2026)	LLMを自律的に機能させるための設計	<i>long-running application development</i>
LangChain(2026)	LLM以外の全要素	<i>Agent = Model + Harness</i>

ハーネスエンジニアリングとは

文献・事例	ハーネスの位置付け	キーワード
Zhang(2025)	LLMに追加可能なモジュール群	<i>modular harness design</i>
Bui(2026)	実行のオーケストレーション層	<i>runtime orchestration layer</i>
Anthropic(2026)	LLMを自律的に機能させるための設計	<i>long-running application development</i>
LangChain(2026)	LLM以外の全要素	<i>Agent = Model + Harness</i>

ハーネスエンジニアリングは、LLMを効果的に動作させるための仕組み全般である「ハーネス」を設計・開発する行為と暫定的に定義できる

■ ハーネスエンジニアリングのスコープ・キーワード

How	Where	When
Principles Verification Tools Skills Orchestration	Workspace Sandbox Runtime Interface Data Sources Permissions	Trigger Event Schedule Queue Approval Points

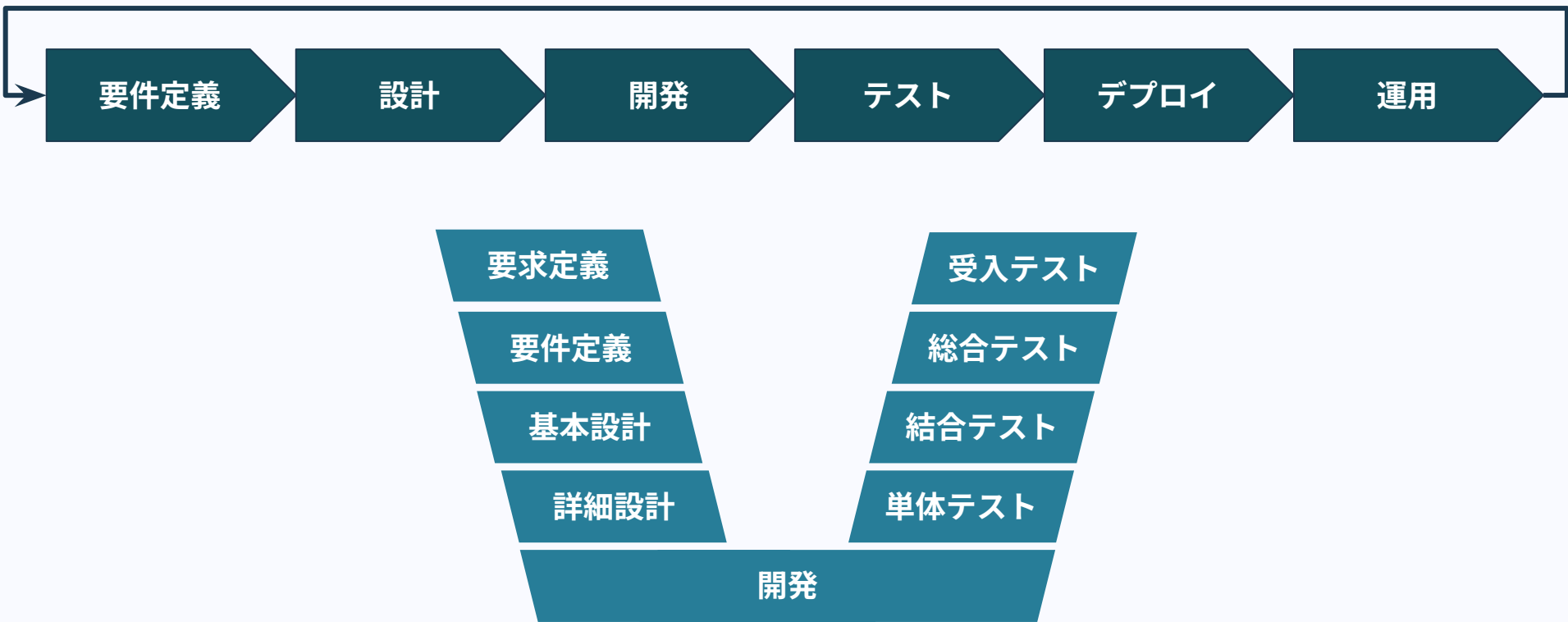
ハーネスエンジニアリングの対象領域は広く、要素は多岐にわたる

ハーネスの構成要素とソフトウェア開発の蓄積

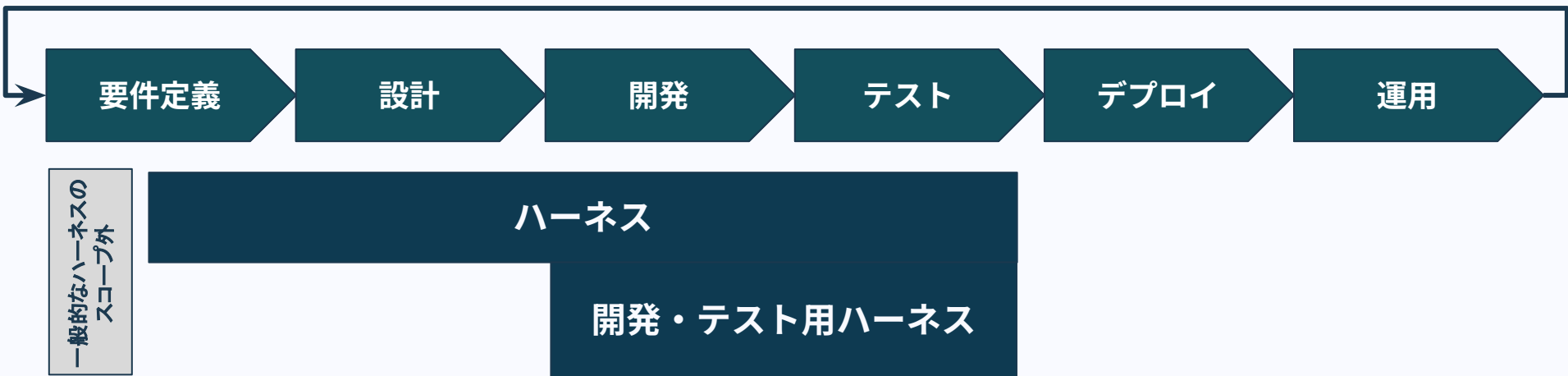
ハーネスの要素	詳細
Principlesの設定	ADRの設定 開発標準(例: TERASOLUNA, HyThology)の適用
Verficationの設定	Lint テストコード CIチェック

ハーネスの構成要素の一部は、これまでのソフトウェア開発の蓄積の上に成り立っている

■ 前提: Software Development Life Cycle(SDLC)とV字モデル



ハーネスの影響範囲と領域



ハーネスはシステムの開発サイクルにおいて大きな影響範囲を持つ。
特に既存のハーネスは開発・テスト領域に集中している。

ハーネスの設計思想

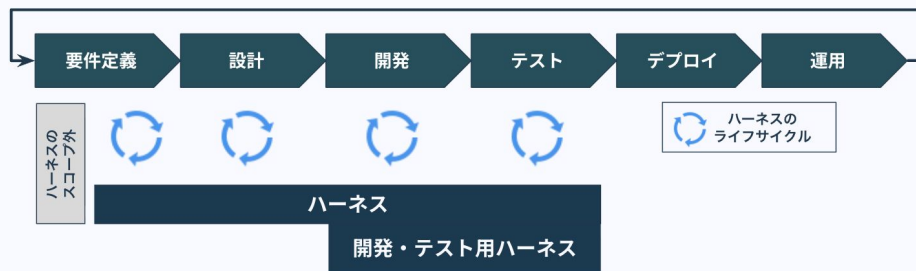
ハーネスエンジニアリングの進め方一例

QCD	指標例	ハーネスの要素例	開発自由度	難易度
D	開発スピード	Skillsの設定	下がる	相対的に低い
Q	要件達成度	Hooksの設定	下がる	相対的に低い
	保守性・変更容易性	Verificationの追加	下がる	相対的に低い
		Principlesの設定	下がる	相対的に低い
C	人的コスト	自律性向上	上がる	相対的に高い
	システムコスト	パラメータチューニング	-	相対的に低い

注力指標を考慮しつつ、システム開発の自由度を下げる施策から始めると
難易度観点と影響範囲の観点から進めやすい

ハーネスエンジニアリングで十分？？？

■ ハーネスの影響範囲と領域



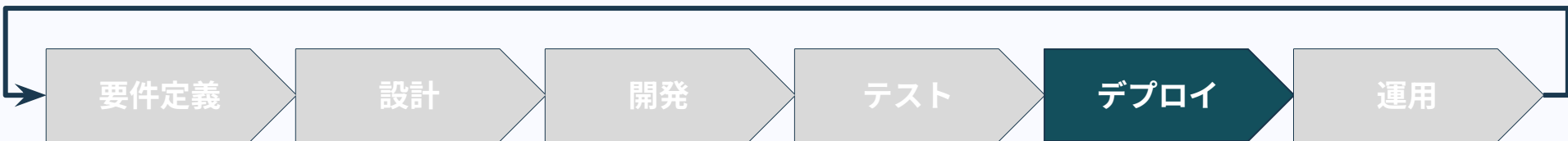
ハーネスはシステムの開発サイクルにおいて大きな影響範囲を持つ。
特に既存のハーネスは開発・テスト領域に集中している。

■ ハーネスエンジニアリングの進め方一例

QCD	指標例	ハーネスの要素	開発自由度	難易度
D	開発スピード	Skillsの設定	下がる	相対的に低い
Q	要件達成度	Hooksの設定	下がる	相対的に低い
	保守性・変更容易性	Verificationの追加	下がる	相対的に低い
		Principlesの設定	下がる	相対的に低い
C	人的コスト	自律性向上	上がる	相対的に高い
	システムコスト	パラメータチューニング	-	相対的に低い

注力指標を考慮しつつ、システム開発の自由度を下げる施策から始めると
難易度観点と影響範囲の観点から進めやすい

■ ハーネスエンジニアリングの不足分(1/2): Four Keys・ボトルネック



Four Keys: ソフトウェア開発チームのパフォーマンス計測指標

デプロイ頻度	変更失敗率 (デプロイ後にバグ等で即時介入が必要となった割合)
変更のリードタイム (コミットからデプロイまでの時間)	デプロイ失敗からの復旧時間

開発・テストが早くなっても、デプロイ頻度を上げられなければ
ユーザーへの価値提供が加速しない

ハーネスエンジニアリングの不足分(2/2): 人の働き方

SPACE: 開発組織を5次元でとらえた指標

満足度・ウェルビーイング

成果

(期待されるアウトカムをどれだけ達成したか)

活動量

(コミット、PRなどの開発量)

コミュニケーション・協働

効率・フロー

ハーネスエンジニアリングはあくまでLLMが働きやすい仕組みづくりであり、人が働きやすい環境を作ることも中長期的な目線で必要である。

ハーネスエンジニアリングの設計ポイント

LLMのためのハーネスを作ると同時に人・チームのための仕組みを整備する必要がある。

ハーネスエンジニアリングの進め方一例

QCD	指標例	ハーネスの要素	開発自由度	難易度
D	開発スピード	Skillsの設定	下がる	相対的に低い
Q	要件達成度	Hooksの設定	下がる	相対的に低い
	保守性・変更容易性	Verificationの追加	下がる	相対的に低い
C	人的コスト	Principlesの設定	下がる	相対的に低い
	システムコスト	自律性向上	上がる	相対的に高い
		パラメータチューニング	-	相対的に低い

注力指標を考慮しつつ、システム開発の自由度を下げる施策から始めると難易度観点と影響範囲の観点から進めやすい

ハーネスエンジニアリングの不足分(1/2): Four Keys・ボトルネック



Four Keys: ソフトウェア開発チームのパフォーマンス計測指標

デプロイ頻度	変更失敗率 (デプロイ後にバグ等で即時介入が必要となった割合)
変更のリードタイム (コミットからデプロイまでの時間)	デプロイ失敗からの復旧時間

開発・テストが早くなっても、デプロイ頻度を上げられなければユーザーへの価値提供が加速しない

ハーネスエンジニアリングはあくまでLLMが働きやすい仕組みづくりであり、人が働きやすい環境を作ることの中長期的な目線で必要である。

システム開発を安全かつ高速にする

開発組織が押さえるべき

AI駆動開発と

ガバナンスの要諦

5.21木
12:00-13:00 **ONLINE**

株式会社ALGOMATIC
AI駆動開発センター
センター長・AI/MLエンジニア
渋谷 優介

株式会社ALGOMATIC
AXガバナンスセンター
シニアエキスパート
独立行政法人情報処理推進機構
専門委員
宮脇 峻平

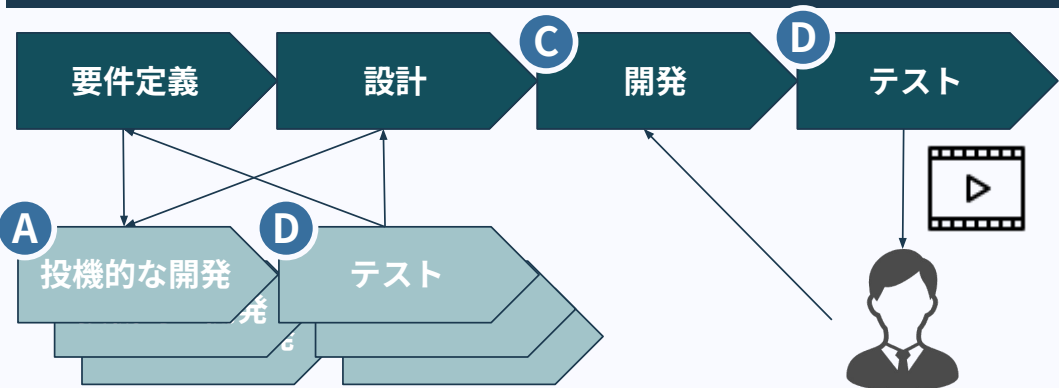
ALGOMATIC
AI Transformation

<https://peatix.com/event/4961703/view>

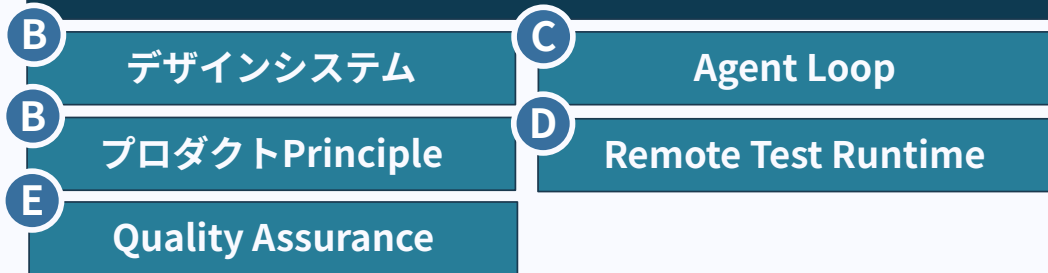
ハーネスエンジニアリングの 事例紹介

Algomaticのハーネスの事例

概念図



ハーネス



解説

- A** 並列開発によるdiscovery業務高度化
- B** ユーザー体験原則を含むデザインシステム
・ 事業KPI含むプロダクトPrincipleを用いた確度の高い開発
- C** 人の介入を模した
Agent Loop機構による開発を自律化
- D** テスト前倒しによる観点潰し込み。
クラウド環境でのテスト自動化によりテスト
負荷軽減。テスト動画をLinearへ添付。
- E** 弊社独自の品質管理基準による
開発機能種別に応じたレビュー

